



Université de Carthage

INSTITUT DES HAUTES ÉTUDES COMMERCIALES DE CARTHAGE

Projet de Conception Orientée Objet(COO) **Conception et réalisation d'une plateforme de covoiturage en Java**

Établi par :

- Hassan Achour
- Ela Slama
- Walid Saheb Ettaba
- Ahmed Zribi
- Ahmed Chermiti

Encadré par:

- Dr. Jihene Tounsi
- Prof. Maroua Laabidi

Année universitaire : 2024 – 2025

RÉSUMÉ

Dans le cadre du module de **conception orientée objet (COO)**, ce projet consiste à concevoir et développer une **plateforme de covoiturage en Java** permettant la mise en relation entre conducteurs et passagers.

L'objectif principal est de mettre en pratique les concepts fondamentaux de la programmation orientée objet, notamment la modélisation UML, l'héritage, l'encapsulation et la structuration d'une application logicielle.

L'application développée permet aux conducteurs de créer et gérer des trajets, et aux passagers de rechercher et réserver des trajets disponibles. Le projet couvre l'ensemble des étapes du cycle de développement logiciel : analyse des besoins, conception, implémentation, tests et perspectives d'amélioration.

Mots-clés : *Covoiturage, Java, UML, Programmation Orientée Objet, Conception logicielle.*

TABLE DES MATIÈRES

1. Introduction
 - 1.1 Contexte
 - 1.2 Problématique
 - 1.3 Objectifs
2. Présentation générale de la solution
3. Analyse des besoins
 - 3.1 Identification des acteurs
 - 3.2 Besoins fonctionnels
 - 3.3 Besoins non fonctionnels
4. Diagramme de cas d'utilisation
 - 4.1 Diagramme global
 - 4.2 Diagramme raffiné
5. Diagramme de classes
 - 5.1 Présentation des classes
 - 5.2 Relations entre les classes
6. Architecture de l'application
 - 6.1 Architecture générale
 - 6.2 Description des couches
7. Implémentation de l'application
 - 7.1 Environnement de développement
 - 7.2 Description des classes principales
 - 7.3 Description des méthodes principales
8. Interface de l'application
 - 8.1 Interface utilisateur
 - 8.2 Scénarios d'utilisation
9. Tests et validation
 - 9.1 Stratégie de test
 - 9.2 Résultats des tests
10. Limites et perspectives
11. Conclusion

INTRODUCTION

1.1 Contexte:

L'essor des technologies de l'information et de la communication a profondément transformé les modes d'organisation des services de transport. Les applications numériques occupent aujourd'hui une place centrale dans la gestion des déplacements, en proposant des solutions visant à optimiser les ressources disponibles et à faciliter la mise en relation entre les usagers. Dans ce contexte, le covoiturage apparaît comme une réponse pertinente aux problématiques économiques, sociales et organisationnelles liées à la mobilité. Sur le plan académique, le développement d'une application de covoiturage constitue un support d'étude particulièrement adapté à l'apprentissage de la programmation orientée objet. Ce type de système repose sur l'identification d'acteurs distincts, la manipulation de données structurées et la définition de règles de gestion claires. Le projet présenté dans ce rapport s'inscrit dans le cadre du module de **conception orientée objet (COO)** de la deuxième année de licence et vise à mettre en pratique les concepts théoriques abordés durant la formation à travers une réalisation concrète en langage Java.

1.2 Problématique:

La conception d'une plateforme de covoiturage ne se limite pas à l'implémentation de fonctionnalités techniques. Elle suppose avant tout une réflexion approfondie sur la structuration du système et sur les interactions entre ses différents composants. La gestion des utilisateurs, des trajets et des réservations doit être pensée de manière cohérente afin d'éviter les redondances, les incohérences et les difficultés de maintenance. Dans un cadre pédagogique, cette exigence est renforcée par la nécessité de respecter les principes de la programmation orientée objet. Une conception approximative peut conduire à un code difficile à comprendre et à faire évoluer, même si l'application semble fonctionner correctement. La problématique centrale de ce projet consiste donc à déterminer comment concevoir et réaliser, en Java, une application de covoiturage reposant sur une modélisation orientée objet rigoureuse, tout en répondant à un besoin fonctionnel réaliste.

1.3 Objectifs:

L'objectif principal de ce projet est de concevoir et de développer une application Java permettant de gérer une plateforme de covoiturage mettant en relation des conducteurs et des passagers. Ce travail vise à illustrer de manière concrète l'application des concepts fondamentaux de la programmation orientée objet, notamment l'encapsulation, l'héritage et les relations entre classes. Ce projet a également pour finalité de développer une démarche méthodologique structurée, allant de l'analyse des besoins à l'implémentation et à la validation de la solution proposée. À travers cette réalisation, il s'agit de démontrer la capacité à modéliser un système logiciel à l'aide de diagrammes UML, à justifier les choix de conception effectués et à produire une application fonctionnelle, conforme aux exigences académiques du module.

CHAPITRE 2 - PRÉSENTATION GÉNÉRALE DE LA SOLUTION

Cette section a pour objectif de présenter de manière globale la solution développée, sans entrer dans les détails techniques qui feront l'objet des chapitres suivants. Il s'agit de donner au lecteur une vision d'ensemble du système proposé, de son fonctionnement général et des acteurs concernés, afin de faciliter la compréhension de la suite du rapport.

La solution proposée prend la forme d'une **application Java de covoiturage**, conçue pour permettre la mise en relation entre des conducteurs proposant des trajets et des passagers souhaitant effectuer des déplacements partagés. L'application repose sur une logique orientée objet, dans laquelle chaque entité du système est représentée par une classe clairement définie, reflétant son rôle et ses responsabilités.

Le système distingue deux catégories principales d'utilisateurs : les conducteurs et les passagers. Les conducteurs ont pour rôle de proposer des trajets en précisant les informations essentielles telles que le lieu de départ, la destination, la date, l'horaire et le tarif. Les passagers, quant à eux, peuvent consulter les trajets disponibles, effectuer des recherches selon leurs besoins et réserver une place sur un trajet existant. Cette distinction entre acteurs constitue un élément central de la conception de l'application et structure l'ensemble des fonctionnalités proposées.

L'application a été conçue de manière à assurer une interaction claire et fluide entre les utilisateurs et le système. Chaque action effectuée par un utilisateur correspond à un traitement précis, garantissant la cohérence des données et le respect des règles définies. Les fonctionnalités offertes sont volontairement limitées à l'essentiel, afin de répondre aux objectifs pédagogiques du projet et de privilégier la qualité de la conception logicielle.

D'un point de vue fonctionnel, la plateforme permet la gestion complète du cycle de covoiturage, depuis la création d'un trajet jusqu'à l'acceptation des réservations. L'utilisateur peut ainsi suivre l'évolution de ses actions, consulter les informations pertinentes et interagir avec le système de manière structurée. Cette organisation vise à illustrer concrètement l'apport de la programmation orientée objet dans la conception d'applications réelles.

Cette présentation générale permet de situer la solution dans son ensemble. Les chapitres suivants seront consacrés à l'analyse détaillée des besoins, à la modélisation du système à l'aide de diagrammes UML, puis à l'implémentation et à la validation de l'application.

CHAPITRE 3 - ANALYSE DES BESOINS

Ce chapitre a pour objectif d'identifier et de formaliser les besoins auxquels doit répondre la plateforme de covoiturage développée. L'analyse des besoins constitue une étape essentielle dans le processus de conception d'un système logiciel, car elle permet de traduire une idée générale en exigences précises et exploitables. Elle sert également de base à la modélisation UML et à l'implémentation de l'application.

L'analyse est organisée en trois parties. La première consiste à identifier les acteurs interagissant avec le système. La deuxième précise les besoins fonctionnels que l'application doit satisfaire. Enfin, la troisième présente les besoins non fonctionnels, qui définissent les contraintes de qualité du système.

3.1 Identification des acteurs:

Un acteur représente une entité externe qui interagit avec le système afin d'atteindre un objectif précis. Dans le cadre de cette application de covoiturage, l'analyse du fonctionnement global du système permet d'identifier deux acteurs principaux, correspondant aux profils d'utilisateurs définis dans la conception et l'implémentation.

Le premier acteur est le **conducteur**. Il s'agit d'un utilisateur qui propose des trajets sur la plateforme. Le conducteur interagit avec le système afin de créer des trajets, de consulter les demandes de réservation associées à ses trajets et de gérer les informations liées à ces derniers. Son rôle est central dans le fonctionnement de la plateforme, puisqu'il est à l'origine de l'offre de covoiturage.

Le second acteur est le **passager**. Cet utilisateur utilise la plateforme pour rechercher des trajets disponibles correspondant à ses besoins de déplacement. Il peut consulter les informations relatives aux trajets proposés par les conducteurs et effectuer des réservations. Le passager interagit donc avec le système principalement dans une logique de consultation et de demande.

Ces deux acteurs disposent de fonctionnalités distinctes, mais complémentaires. Leur identification claire permet de structurer les fonctionnalités du système et de faciliter la conception des diagrammes de cas d'utilisation présentés dans le chapitre suivant.

3.2 Besoins fonctionnels:

Les besoins fonctionnels décrivent les services que le système doit fournir aux acteurs. Ils correspondent aux actions que les utilisateurs peuvent effectuer à travers l'application et définissent le comportement attendu du système.

Pour le conducteur, le système doit permettre l'inscription et l'authentification afin de garantir un accès sécurisé aux fonctionnalités réservées. Une fois connecté, le conducteur doit être en mesure de créer un trajet en renseignant les informations nécessaires, telles que le lieu de départ, la destination, la date, l'horaire et le tarif. Le système doit également lui permettre de

consulter la liste des trajets qu'il a créés et d'en modifier certains paramètres lorsque cela est nécessaire. Enfin, le conducteur doit pouvoir consulter les demandes de réservation associées à ses trajets et visualiser les passagers acceptés.

Pour le passager, le système doit permettre l'inscription et l'authentification afin d'accéder aux fonctionnalités de recherche et de réservation. Le passager doit pouvoir consulter l'ensemble des trajets disponibles et effectuer des recherches en fonction de critères spécifiques. Une fois un trajet sélectionné, il doit être possible d'effectuer une réservation. Le système doit également permettre au passager de consulter ses réservations, notamment celles qui ont été acceptées par les conducteurs.

L'ensemble de ces fonctionnalités constitue le cœur du fonctionnement de la plateforme de covoiturage. Elles sont directement issues de l'analyse du besoin initial et sont cohérentes avec les fonctionnalités implémentées dans l'application Java.

3.3 Besoins non fonctionnels:

Les besoins non fonctionnels définissent les contraintes de qualité auxquelles le système doit répondre. Contrairement aux besoins fonctionnels, ils ne décrivent pas ce que fait le système, mais comment il doit le faire.

Dans le cadre de ce projet, l'application doit avant tout être simple à utiliser. L'interface proposée doit permettre aux utilisateurs de comprendre rapidement les actions possibles et de naviguer facilement entre les différentes fonctionnalités. Cette exigence est particulièrement importante dans un contexte pédagogique, où la clarté du fonctionnement prime sur la complexité.

La fiabilité du système constitue également un besoin non fonctionnel essentiel. Les données relatives aux utilisateurs, aux trajets et aux réservations doivent être cohérentes et correctement gérées tout au long de l'exécution de l'application. Le système doit éviter les incohérences susceptibles de nuire à son bon fonctionnement.

Enfin, l'application doit être conçue de manière à être maintenable et évolutive. La structuration du code en classes bien définies et l'utilisation des principes de la programmation orientée objet facilitent les évolutions futures, telles que l'ajout de nouvelles fonctionnalités ou l'intégration d'un mode de persistance plus avancé.

CHAPITRE 4 - DIAGRAMME DE CAS D'UTILISATION

Ce chapitre est consacré à la modélisation fonctionnelle du système à l'aide des diagrammes de cas d'utilisation UML. Ces diagrammes permettent de représenter les interactions entre les acteurs et la plateforme de covoiturage, en mettant en évidence les fonctionnalités offertes par l'application. Ils constituent une étape essentielle du processus de conception, située entre l'analyse des besoins et la conception structurelle du système.

La modélisation des cas d'utilisation repose directement sur les besoins fonctionnels identifiés précédemment. Elle permet de formaliser ces besoins de manière claire et structurée, tout en assurant une compréhension globale du comportement du système.

4.1 Diagramme de cas d'utilisation global:

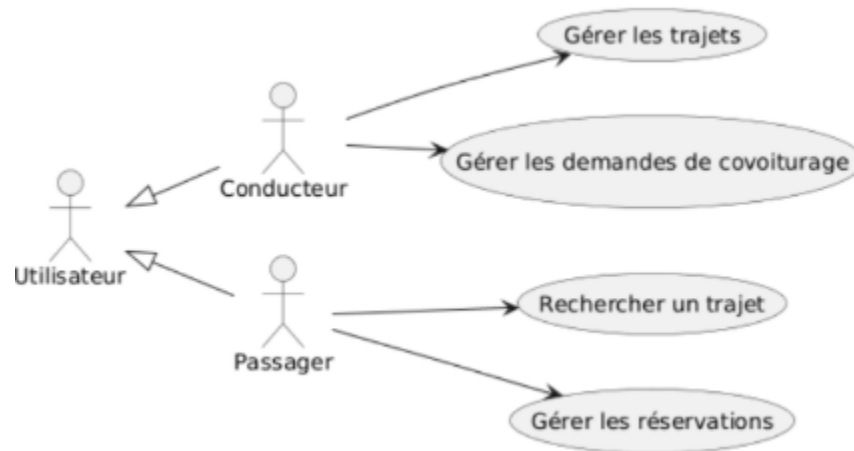


Figure 4.1 : Diagramme de cas d'utilisation global de la plateforme de covoiturage

Le diagramme de cas d'utilisation global offre une vision d'ensemble des interactions entre les acteurs et le système. Il met en évidence les deux acteurs principaux identifiés, à savoir le conducteur et le passager, ainsi que l'ensemble des fonctionnalités majeures de la plateforme de covoiturage.

Dans ce diagramme, le conducteur est associé aux cas d'utilisation relatifs à la gestion des trajets. Il peut s'inscrire et s'authentifier afin d'accéder aux fonctionnalités réservées, créer des trajets, consulter les trajets qu'il a proposés, modifier certaines informations associées à ces trajets et consulter les demandes de réservation ainsi que les passagers acceptés.

Le passager est associé aux cas d'utilisation liés à la consultation et à la réservation de trajets. Après authentification, il peut consulter les trajets disponibles, effectuer des recherches en

fonction de ses besoins et réserver une place sur un trajet existant. Il peut également consulter l'état de ses réservations.

Ce diagramme permet de délimiter clairement le périmètre fonctionnel du système et de distinguer les responsabilités de chaque acteur, tout en offrant une vue synthétique du fonctionnement global de l'application.

4.2 Diagramme de cas d'utilisation raffiné:

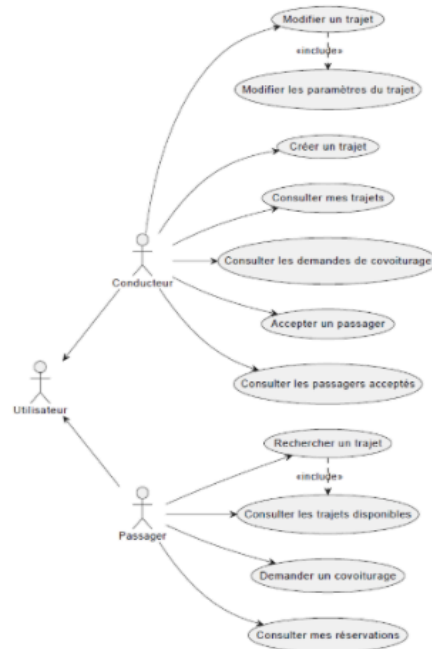


Figure 4.2 : Diagramme de cas d'utilisation raffiné de la plateforme de covoiturage

Le diagramme de cas d'utilisation raffiné a pour objectif de détailler les interactions présentées dans le diagramme global. Il permet de représenter de manière plus précise les actions spécifiques réalisées par chaque acteur et les différentes étapes associées à ces actions.

Pour le conducteur, le diagramme raffiné met en évidence le processus de gestion des trajets. Une fois authentifié, le conducteur peut créer un trajet en renseignant les informations nécessaires, consulter la liste de ses trajets et modifier certains paramètres, tels que le tarif ou les horaires. Le diagramme illustre également la consultation des demandes de réservation et la visualisation des passagers acceptés.

Pour le passager, le diagramme raffiné détaille le processus de recherche et de réservation de trajets. Il montre comment le passager peut consulter les trajets disponibles, effectuer une recherche ciblée, réserver un trajet et consulter ses réservations acceptées.

La distinction entre le diagramme global et le diagramme raffiné permet d'obtenir une compréhension progressive et détaillée du fonctionnement du système. Cette modélisation facilite la transition vers la conception statique de l'application, qui sera abordée dans le chapitre suivant à travers le diagramme de classes.

CHAPITRE 5 - DIAGRAMME DE CLASSES

Ce chapitre est consacré à la présentation du diagramme de classes UML de la plateforme de covoiturage. Le diagramme de classes permet de représenter la structure statique du système en identifiant les différentes classes, leurs attributs, leurs méthodes ainsi que les relations existantes entre elles. Il constitue une étape centrale de la conception orientée objet, car il sert de lien entre l'analyse fonctionnelle et l'implémentation de l'application.

La conception du diagramme de classes repose directement sur les besoins fonctionnels identifiés précédemment et sur la modélisation des cas d'utilisation. Elle vise à assurer une organisation cohérente du système, facilitant la compréhension, la maintenance et l'évolution future de l'application.

5.1 Présentation des classes:

L'analyse du système a permis d'identifier plusieurs classes principales représentant les entités essentielles de la plateforme de covoiturage. Ces classes correspondent aux acteurs du système et aux objets métiers manipulés par l'application.

La classe **User** représente l'utilisateur de manière générale. Elle regroupe les informations communes à tous les utilisateurs de la plateforme, telles que les données d'identification et les informations personnelles. Cette classe constitue une base commune permettant de factoriser les attributs et les comportements partagés par les différents types d'utilisateurs.

La classe **Conducteur** hérite de la classe User et représente un utilisateur proposant des trajets. Elle contient des informations spécifiques au rôle de conducteur, notamment celles liées à la gestion des trajets proposés et aux paramètres associés à ces trajets. Les méthodes de cette classe permettent au conducteur de créer et de gérer ses trajets.

La classe **Passager** hérite également de la classe User. Elle représente un utilisateur recherchant et réservant des trajets. Cette classe permet de gérer les actions liées à la consultation des trajets disponibles et au suivi des réservations effectuées par le passager.

La classe **Trajet** représente un élément central du système. Elle regroupe les informations relatives à un déplacement proposé, telles que le lieu de départ, la destination, la date, l'horaire, le nombre de places disponibles et le tarif. Cette classe permet de modéliser l'offre de covoiturage et d'assurer le lien entre les conducteurs et les passagers.

L'ensemble de ces classes a été conçu dans le respect des principes de la programmation orientée objet, en veillant à une séparation claire des responsabilités et à une structuration cohérente du système.

5.2 Relations entre les classes:

Les relations entre les classes permettent de traduire les interactions structurelles existant au sein du système. Elles sont essentielles pour comprendre le fonctionnement global de l'application et la manière dont les différentes entités collaborent.

Le diagramme de classes met en évidence une relation d'héritage entre la classe User et les classes Conducteur et Passager. Cette relation permet de mutualiser les attributs et les méthodes communs tout en spécialisant le comportement de chaque type d'utilisateur selon son rôle dans la plateforme.

Une relation d'association existe entre la classe Conducteur et la classe Trajet. Cette association traduit le fait qu'un conducteur peut proposer un ou plusieurs trajets. Chaque trajet est ainsi lié à un conducteur responsable de sa création et de sa gestion.

Une autre relation d'association relie la classe Passager à la classe Trajet. Cette relation permet de représenter la sélection ou la réservation d'un trajet par un passager. Elle traduit l'interaction fonctionnelle entre les passagers et les trajets proposés sur la plateforme.

L'ensemble de ces relations est représenté dans le diagramme de classes afin de fournir une vision claire et structurée de l'architecture statique du système. Cette modélisation facilite la compréhension du code source et garantit une correspondance directe entre la conception UML et l'implémentation de l'application.

La Figure 5.1 illustre le diagramme de classes de la plateforme de covoiturage et synthétise les différentes classes identifiées ainsi que les relations existantes entre elles.

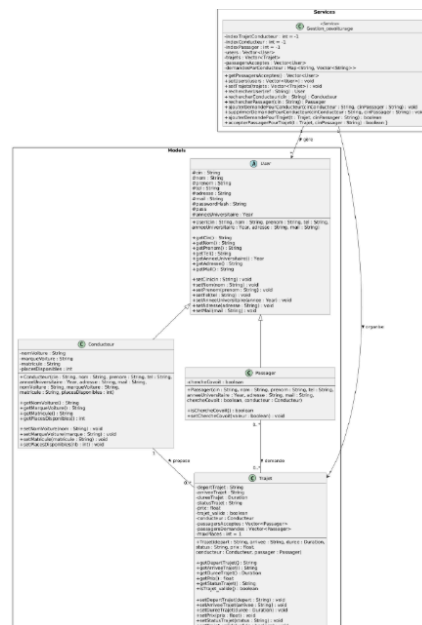


Figure 5.1 : Diagramme de classes de la plateforme de covoiturage

CHAPITRE 6 - ARCHITECTURE DE L'APPLICATION

Ce chapitre présente l'architecture générale de la plateforme de covoiturage développée. L'architecture d'une application logicielle définit l'organisation globale de ses composants ainsi que les interactions existantes entre eux. Elle joue un rôle essentiel dans la lisibilité du code, sa maintenabilité et son évolutivité.

Dans le cadre de ce projet, une architecture simple et cohérente a été adoptée afin de répondre aux objectifs pédagogiques du module de **conception orientée objet (COO)**, tout en assurant une séparation claire des responsabilités entre les différentes parties du système.

6.1 Architecture générale:

L'architecture de l'application repose sur une organisation en couches permettant de distinguer clairement les différents rôles assurés par les composants du système. Cette structuration facilite la compréhension globale de l'application et permet de limiter les dépendances directes entre les éléments.

L'application est organisée principalement autour de deux ensembles majeurs : la couche des modèles et la couche des services. La couche des modèles regroupe les classes représentant les entités métier du système, telles que les utilisateurs et les trajets. La couche des services, quant à elle, assure la gestion des traitements applicatifs et la coordination des interactions entre ces entités.

Cette organisation permet de centraliser les règles de gestion dans des composants dédiés, tout en conservant des classes métier simples et clairement définies. Elle favorise ainsi une correspondance directe entre la conception UML et l'implémentation en Java.

La vue d'ensemble de cette organisation est illustrée par le diagramme présentant la séparation entre la couche des services et la couche des modèles. Ce diagramme permet de visualiser la manière dont les traitements applicatifs interagissent avec les entités métier du système et met en évidence la structure générale adoptée pour l'application.

Figure 5.1 : Diagramme de classes de la plateforme de covoiturage

6.2 Description des couches:

La couche des modèles constitue le cœur métier de l'application. Elle regroupe les classes représentant les entités principales du système, telles que les utilisateurs et les trajets. Ces classes contiennent les attributs et les méthodes nécessaires à la manipulation des données, sans inclure de logique de traitement complexe. Cette approche permet de maintenir une séparation nette entre les données et les traitements.

La couche des services est chargée de gérer la logique applicative et les interactions entre les différentes entités du système. Elle assure notamment la création et la gestion des trajets, le traitement des demandes de covoiturage ainsi que la coordination des actions entre les conducteurs et les passagers. En centralisant ces traitements, la couche de services permet de réduire les dépendances directes entre les classes métier et d'améliorer la maintenabilité du code.

Cette séparation en couches contribue à rendre l'application plus évolutive. Elle permet, par exemple, d'envisager l'ajout de nouvelles fonctionnalités ou la modification du mode de persistance des données sans remettre en cause l'ensemble de la structure existante. L'architecture adoptée répond ainsi aux exigences de clarté, de cohérence et de modularité attendues dans un projet de conception orientée objet.

CHAPITRE 7 - IMPLÉMENTATION DE L'APPLICATION

Ce chapitre est consacré à l'implémentation de la plateforme de covoiturage à l'aide du langage Java. Après les phases d'analyse des besoins et de conception UML, l'implémentation constitue l'étape au cours de laquelle les choix de modélisation sont traduits en code exécutable. L'objectif principal de cette phase est de développer une application fonctionnelle respectant les principes de la programmation orientée objet et la structure définie dans les chapitres précédents.

L'implémentation a été réalisée de manière progressive, en veillant à assurer une correspondance directe entre les diagrammes UML et les classes Java développées. Cette approche permet de garantir la cohérence globale du système et de faciliter la compréhension du code.

7.1 Environnement de développement:

L'application a été développée en utilisant le langage **Java**, choisi pour sa robustesse et son adéquation avec les principes de la programmation orientée objet. Le développement a été réalisé dans un environnement de développement intégré (IDE) permettant la gestion des classes, la compilation du code et l'exécution de l'application.

L'organisation du projet respecte une structuration claire des fichiers source, facilitant la lecture et la maintenance du code. Chaque classe est définie dans un fichier distinct et regroupe les attributs et les méthodes correspondant à son rôle dans le système.

7.2 Implémentation des classes principales:

L'implémentation de l'application repose sur les classes principales identifiées dans le diagramme de classes. Chaque classe a été développée conformément à son rôle défini lors de la phase de conception.

La classe **User** constitue la classe de base représentant un utilisateur de la plateforme. Elle regroupe les attributs communs, tels que les informations d'identification, ainsi que les méthodes permettant l'accès et la modification de ces données. Cette classe permet de factoriser les comportements communs aux différents types d'utilisateurs.

Les classes **Conducteur** et **Passager** héritent de la classe User. Cette relation d'héritage permet de spécialiser le comportement de chaque type d'utilisateur tout en conservant les attributs communs. La classe Conducteur implémente les fonctionnalités liées à la création et à la gestion des trajets, tandis que la classe Passager permet la consultation des trajets disponibles et la gestion des réservations.

La classe **Trajet** représente l'entité centrale de l'application. Elle contient les informations relatives à un déplacement proposé, telles que le point de départ, la destination, la date, l'horaire, le nombre de places disponibles et le tarif. Les méthodes associées à cette classe permettent de gérer les données du trajet et d'assurer les interactions avec les utilisateurs.

7.3 Gestion des fonctionnalités:

Les fonctionnalités de l'application ont été implémentées de manière à respecter les scénarios définis dans les diagrammes de cas d'utilisation, de séquence et d'activité. Chaque action réalisée par un utilisateur correspond à une méthode spécifique dans le code Java.

La création d'un trajet par un conducteur est gérée par des méthodes dédiées permettant l'instanciation d'un objet Trajet et son association au conducteur concerné. De même, la recherche et la réservation d'un trajet par un passager sont traitées par des méthodes assurant la consultation des trajets disponibles et la gestion des demandes de covoiturage.

L'enchaînement des actions lors d'un scénario d'utilisation, tel que la réservation d'un trajet, respecte la logique définie dans les diagrammes de séquence et d'activité. Cette correspondance garantit une implémentation fidèle à la conception UML et facilite la validation du comportement de l'application.

7.4 Gestion des données:

Dans le cadre de ce projet, la gestion des données est assurée de manière simple, sans recours à une base de données externe. Les informations relatives aux utilisateurs et aux trajets sont manipulées directement à travers les objets Java, ce qui permet de se concentrer sur les aspects de conception et d'implémentation orientée objet.

Cette approche est adaptée au contexte pédagogique du projet et permet de limiter la complexité technique, tout en illustrant clairement les interactions entre les différentes entités du système.

7.5 Validation de l'implémentation:

L'implémentation de l'application a été validée par des tests fonctionnels réalisés lors de l'exécution du programme. Ces tests ont permis de vérifier le bon fonctionnement des principales fonctionnalités, notamment la création de trajets, la recherche de trajets disponibles et la gestion des réservations.

La conformité entre l'implémentation et la conception UML constitue un critère essentiel de validation. Les classes, les relations et les interactions observées dans le code correspondent aux éléments définis dans les diagrammes présentés dans les chapitres précédents, ce qui confirme la cohérence globale du système.

7.6 Limites de l'implémentation:

Malgré son bon fonctionnement, l'application présente certaines limites liées au périmètre du projet. L'absence de persistance avancée des données et d'interface graphique évoluée constitue une contrainte volontaire, liée aux objectifs pédagogiques et au temps imparti pour la réalisation du projet.

Ces limites n'affectent pas la validité de l'implémentation, mais ouvrent des perspectives d'amélioration qui pourront être envisagées dans la suite du travail.

CHAPITRE 8 - INTERFACE DE L'APPLICATION

Ce chapitre présente l'interface de l'application ainsi que les scénarios d'utilisation associés. Il a pour objectif de décrire la manière dont les utilisateurs interagissent avec la plateforme de covoiturage et de mettre en évidence le comportement dynamique du système lors de l'exécution des principales fonctionnalités.

L'analyse de l'interface et des scénarios permet de compléter la conception statique présentée dans les chapitres précédents, en apportant une vision temporelle et fonctionnelle des interactions entre les acteurs et le système.

8.1 Interface utilisateur:

L'interface de l'application permet aux utilisateurs d'accéder aux fonctionnalités de la plateforme en fonction de leur rôle. Après authentification, l'utilisateur peut agir soit en tant que conducteur, soit en tant que passager, et accéder aux fonctionnalités correspondantes.

Pour le conducteur, l'interface permet notamment la création de trajets, la consultation des trajets proposés ainsi que la gestion des demandes de covoiturage reçues. Cette organisation facilite le suivi des trajets et la prise de décision concernant l'acceptation ou le refus des passagers.

Pour le passager, l'interface permet la recherche de trajets disponibles, la consultation des informations associées à chaque trajet et l'envoi de demandes de covoiturage. Le passager peut également consulter l'état de ses réservations, ce qui permet un suivi clair des trajets acceptés.

L'interface a été conçue de manière à assurer une navigation simple et cohérente, tout en respectant la logique définie lors de la phase de conception.

8.2 Scénarios d'utilisation:

Les scénarios d'utilisation décrivent le déroulement des interactions entre les utilisateurs et le système lors de l'exécution des fonctionnalités principales de l'application. Ils permettent d'analyser le comportement dynamique du système et de vérifier la cohérence entre la conception UML et l'implémentation.

Un scénario représentatif est celui de la réservation d'un trajet par un passager. Dans ce scénario, le passager recherche un trajet correspondant à ses besoins, consulte les trajets disponibles et envoie une demande de covoiturage pour le trajet sélectionné. Le système traite ensuite cette demande et met à jour les informations associées au trajet concerné.

Le déroulement temporel de ce scénario est illustré par le diagramme de séquence. Ce diagramme met en évidence l'ordre des échanges entre les différents acteurs et composants du système, notamment le passager, le conducteur et le système. Il permet de visualiser les messages échangés et les traitements effectués lors de la réservation d'un trajet.

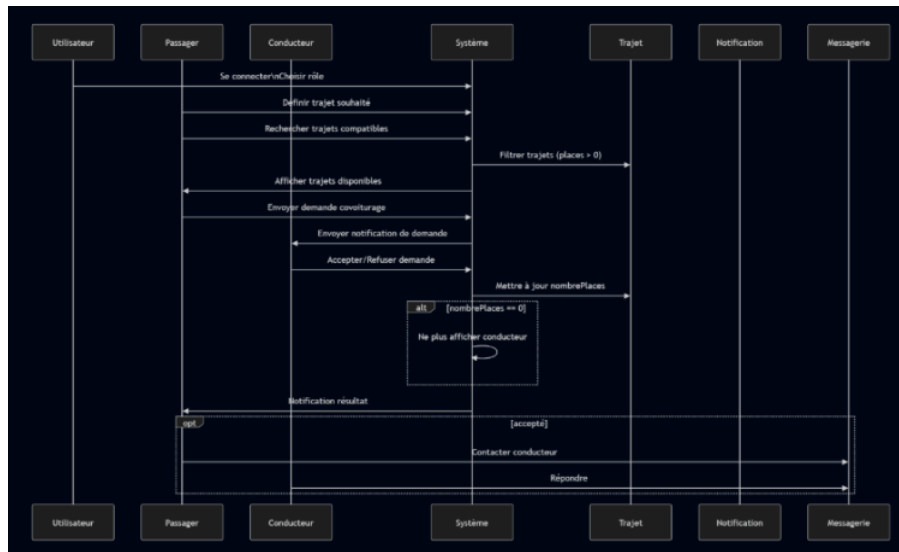


Figure 8.1 : Diagramme de séquence du processus de réservation d'un trajet

Le fonctionnement global du processus de covoiturage est également représenté à l'aide d'un diagramme d'activité. Ce diagramme permet de décrire l'enchaînement des actions et des décisions, depuis la recherche d'un trajet jusqu'à la confirmation ou le refus de la demande de covoiturage. Il met en évidence les différentes étapes du processus ainsi que les choix possibles effectués par l'utilisateur ou le système.



Figure 8.2 : Diagramme d'activité du processus de covoiturage

L'utilisation conjointe du diagramme de séquence et du diagramme d'activité permet d'obtenir une vision complète du comportement dynamique de l'application. Ces diagrammes complètent la conception statique présentée dans le diagramme de classes et assurent une cohérence globale entre la modélisation UML et l'implémentation Java.

8.3 Validation des scénarios:

Les scénarios d'utilisation ont été validés lors de l'exécution de l'application. Les tests réalisés ont permis de vérifier que les interactions prévues par les diagrammes de séquence et d'activité sont correctement implémentées et que les fonctionnalités répondent aux besoins fonctionnels définis précédemment.

Les résultats obtenus confirment que l'application permet une interaction correcte entre les utilisateurs et le système, aussi bien du point de vue du conducteur que du passager. Cette validation garantit la cohérence entre la conception UML, l'implémentation et l'utilisation réelle de l'application.

CHAPITRE 9 - TESTS ET VALIDATION

Ce chapitre est consacré à la phase de tests et de validation de la plateforme de covoiturage développée. Les tests constituent une étape essentielle du cycle de développement d'une application logicielle, car ils permettent de vérifier la conformité du système par rapport aux besoins fonctionnels définis et d'assurer le bon fonctionnement des différentes fonctionnalités implémentées.

Dans le cadre de ce projet, les tests ont été réalisés après l'implémentation des fonctionnalités principales, afin de valider le comportement global de l'application et la cohérence entre la conception UML et le code Java.

9.1 Stratégie de test:

La stratégie de test adoptée repose principalement sur des tests fonctionnels. Ces tests visent à vérifier que chaque fonctionnalité de l'application se comporte conformément aux besoins fonctionnels identifiés lors de la phase d'analyse.

Les tests ont été réalisés en exécutant l'application et en simulant différents scénarios d'utilisation impliquant les deux types d'utilisateurs : le conducteur et le passager. Chaque fonctionnalité a été testée de manière indépendante afin de s'assurer de son bon fonctionnement avant de valider le système dans son ensemble.

Les principaux éléments testés concernent l'inscription et l'authentification des utilisateurs, la création et la gestion des trajets par le conducteur, la recherche de trajets disponibles par le passager ainsi que la gestion des demandes de covoiturage. Une attention particulière a été portée à l'enchaînement des actions afin de vérifier que le comportement du système correspond aux scénarios définis dans les diagrammes de séquence et d'activité.

Cette approche permet de détecter rapidement les erreurs de logique ou les incohérences dans le traitement des données, tout en restant adaptée au périmètre pédagogique du projet.

9.2 Résultats des tests:

Les tests réalisés ont permis de valider le bon fonctionnement des principales fonctionnalités de la plateforme de covoiturage. Les scénarios d'utilisation testés se sont déroulés conformément aux attentes, aussi bien du point de vue du conducteur que du passager.

La création de trajets par le conducteur a été correctement prise en charge par le système, avec une mise à jour cohérente des informations associées aux trajets. De même, la recherche de trajets disponibles par le passager et l'envoi de demandes de covoiturage ont fonctionné comme prévu. Les interactions entre les utilisateurs et le système respectent la logique définie dans les diagrammes UML.

Les tests ont également permis de vérifier la cohérence des données manipulées par l'application, notamment la gestion du nombre de places disponibles et l'association correcte des

trajets aux utilisateurs concernés. Aucun dysfonctionnement majeur n'a été constaté lors de l'exécution des scénarios testés.

Ces résultats confirment que l'application répond aux besoins fonctionnels définis et que l'implémentation est conforme à la conception réalisée lors des phases précédentes.

CHAPITRE 10 - LIMITES ET PERSPECTIVES

10.1 Limites du projet:

Malgré les résultats obtenus, le projet présente certaines limites liées principalement à son périmètre pédagogique et aux contraintes de temps. La plateforme de covoiturage développée vise avant tout à illustrer les principes de la conception orientée objet et leur mise en œuvre en Java. Cela a conduit à des choix techniques volontairement simples.

L'une des principales limites concerne la gestion des données. L'application ne s'appuie pas sur une base de données pour assurer la persistance des informations relatives aux utilisateurs, aux trajets et aux réservations. Les données sont manipulées directement à travers les objets Java, ce qui limite l'utilisation de l'application à un contexte de démonstration.

Par ailleurs, l'interface proposée reste basique et ne constitue pas une interface graphique avancée. Les aspects liés à l'ergonomie, à l'expérience utilisateur et à l'accessibilité n'ont pas été approfondis. De même, certains éléments tels que la gestion avancée des erreurs, la sécurité des données et l'optimisation des performances n'ont pas été traités de manière exhaustive.

Ces limites n'affectent pas la cohérence du projet, mais reflètent les contraintes liées aux objectifs pédagogiques et au niveau du module.

10.2 Perspectives d'amélioration:

Plusieurs perspectives d'amélioration peuvent être envisagées afin d'enrichir et de faire évoluer la plateforme de covoiturage développée. Une première évolution consisterait à intégrer une base de données afin d'assurer la persistance des données. Cela permettrait de rendre l'application plus robuste et plus adaptée à un usage réel.

L'amélioration de l'interface utilisateur constitue également une perspective importante. La mise en place d'une interface graphique plus évoluée permettrait d'améliorer l'expérience utilisateur et de faciliter l'interaction avec le système. D'autres fonctionnalités pourraient être ajoutées, telles que la gestion des notifications, l'évaluation des utilisateurs ou la recherche avancée de trajets.

Enfin, l'architecture de l'application pourrait être enrichie afin de faciliter son déploiement dans un contexte réel. L'intégration de mécanismes de sécurité, la gestion des performances et l'adoption d'une architecture plus modulaire permettraient d'améliorer la maintenabilité et l'évolutivité du système.

Ces perspectives ouvrent la voie à une évolution future du projet vers une application plus complète et plus adaptée à un environnement professionnel.

CHAPITRE 11 - CONCLUSION

Ce projet avait pour objectif la conception et la réalisation d'une plateforme de covoiturage en utilisant le langage Java et les principes de la programmation orientée objet. Il s'inscrit dans le cadre du module de Conception Orientée Objet et vise à mettre en pratique les notions théoriques abordées au cours de la formation.

La première phase du projet a consisté en l'analyse des besoins, permettant d'identifier clairement les attentes des utilisateurs et de définir les fonctionnalités principales du système. Cette analyse a servi de base à la modélisation du système à l'aide des diagrammes UML, notamment les diagrammes de cas d'utilisation, de classes, de séquence et d'activité. Ces diagrammes ont permis de structurer le système et d'assurer une cohérence entre les différentes composantes de l'application.

La phase d'implémentation a permis de traduire cette conception en une application Java fonctionnelle. Les classes développées respectent les principes fondamentaux de la programmation orientée objet, tels que l'encapsulation, l'héritage et la modularité. Les fonctionnalités principales, comme la création de trajets, la recherche de trajets et la gestion des demandes de covoiturage, ont été implémentées et validées à travers des tests fonctionnels.

Les tests réalisés ont confirmé la conformité de l'application par rapport aux besoins fonctionnels définis. Ils ont également permis de vérifier la cohérence entre la conception UML et l'implémentation, ainsi que le bon fonctionnement global du système.

Ce projet a permis d'acquérir une expérience concrète dans la réalisation d'une application logicielle, en suivant une démarche méthodologique complète allant de l'analyse à la validation. Il a contribué au développement de compétences techniques et méthodologiques essentielles dans le domaine de la conception orientée objet et du développement logiciel.